Reporte Técnico de Actividades Práctico-Experimentales Nro. 006

Validación estadística de números pseudoaleatorios mediante Chi-cuadrado y Kolmogorov-Smirnov

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Emilio Josue Flores Gallardo
Asignatura	Simulación
Ciclo	5
Unidad	U2: Generación de números pseudoaleatorios y generación de variables aleatorias
Resultado de aprendizaje de la unidad	R2. Interpreta los resultados de una Simulación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	006
Título de la Práctica	Validación estadística de números pseudoaleatorios mediante Chi-cuadrado y Kolmogorov-Smirnov
Nombre del Docente	José O. Guamán Q.
Fecha	05 y 06 de noviembre de 2026
Horario	Jueves 10h30 a 11h30 / Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas
Repositorio Git	Repositorio GitHub

2. Objetivo(s) de la Práctica

Objetivo general: evaluar la uniformidad en $[0, 1]$ de una secuencia pseudoaleatoria mediante las pruebas de bondad de ajuste Chi-cuadrado y Kolmogorov-Smirnov.

Objetivos específicos:

- Generar secuencias pseudoaleatorias con el método congruencial lineal (LCG).
- Evaluar la uniformidad de frecuencias observadas mediante la prueba Chi-cuadrado.
- Comparar la distribución acumulada empírica y teórica con la prueba Kolmogorov-Smirnov.
- Interpretar los estadísticos de prueba y sus valores p.
- Comparar los resultados entre las implementaciones en Python y Java.

3. Materiales, Reactivos, Equipos y Herramientas

Software y herramientas:

- **Python 3.10+ (`NumPy`, `Matplotlib`):** generación LCG, pruebas estadísticas y gráficas.
- **Java JDK 17+ (`Swing`, `AWT`):** implementación con enteros de 64 bits (`long`) y gráficas nativas.
- **VS Code y terminal GNU/Linux:** edición, compilación y ejecución.
- **GitHub:** control de versiones y repositorio.

4. Procedimiento / Metodología Ejecutada

4.1. Fundamento teórico

Para validar la uniformidad de una secuencia LCG en $[0, 1]$ con significancia $\alpha = 0.05$, se contrastan las hipótesis:

$$H_0 : X \sim U(0,1) \qquad \text{frente a} \qquad H_1 : X \nsim U(0,1) \tag{1}$$

Si $p \geq \alpha$, no se rechaza H_0; si $p < \alpha$, se rechaza H_0.

1. Generador Congruencial Lineal (LCG):

$$X_{n+1} = (aX_n + c) \bmod m, \qquad U_{n+1} = \frac{X_{n+1}}{m} \tag{2}$$

2. Prueba Chi-cuadrado (χ^2): divide $[0, 1]$ en $k = 10$ subintervalos iguales y compara la frecuencia observada O_i con la esperada $E_i = N/k$ ($\chi^2_{\text{crítico}} \approx 16.9190$ para $\alpha = 0.05$ y $gl = 9$):

$$\chi^2 = \sum_{i=1}^{k} \frac{(O_i - E_i)^2}{E_i}, \qquad gl = k - 1 = 9 \tag{3}$$

3. Prueba Kolmogorov-Smirnov (KS): ordena la muestra ($x_{(1)} \leq \cdots \leq x_{(N)}$) y calcula la máxima discrepancia entre la acumulada empírica $F_n(x_{(i)}) = i/N$ y la teórica $F(x_{(i)}) = x_{(i)}$:

$$D^+ = \max_{1 \leq i \leq N} \left(\frac{i}{N} - x_{(i)} \right), \qquad D^- = \max_{1 \leq i \leq N} \left(x_{(i)} - \frac{i-1}{N} \right), \qquad D = \max(D^+, D^-) \tag{4}$$

Tabla 1: Parámetros del generador LCG y de las pruebas estadísticas.

Parámetro	Valor	Descripción
Semilla inicial (X_0)	12345	Valor inicial del generador.
Multiplicador (a)	1664525	Factor multiplicativo LCG.
Incremento (c)	1013904223	Constante aditiva LCG.
Módulo (m)	$2^{32} = 4294967296$	Límite modular ($1\text{L} \ll 32$ en Java).
Tamaño de muestra (N)	$10\,000$	Total de números generados.
Subintervalos (χ^2, k)	10	Clases de ancho 0.1 ($gl = 9$, $E_i = 1000$).
Significancia (α)	0.05	Umbral de decisión para H_0.

4.2. Las 7 etapas de un proyecto de simulación

- **Etapa 1. Definición del problema:** Validar si $10\,000$ números generados con LCG siguen una distribución $U(0, 1)$ con $\alpha = 0.05$.
- **Etapa 2. Formulación conceptual:** Definir las hipótesis H_0 y H_1, la recurrencia LCG y las fórmulas de los estadísticos χ^2, D y sus valores p.

- **Etapa 3. Recolección y análisis de datos:** Configurar los parámetros base: $X_0 = 12345$, $a = 1664525$, $c = 1013904223$, $m = 2^{32}$, $N = 10\,000$, $k = 10$ y $\alpha = 0.05$.
- **Etapa 4. Construcción del modelo:** Programar el sistema en Python y Java bajo arquitectura MVC separada en cinco paquetes (`numberTreatment`, `model`, `service`, `view`, `controller`).
- **Etapa 5. Verificación y validación:** Comprobar que ambos lenguajes generen los mismos $10\,000$ valores, frecuencias observadas ($\sum O_i = 10\,000$) y estadísticos ($\chi^2 = 14.16$, $D = 0.00596$).
- **Etapa 6. Experimentación y análisis de resultados:** Evaluar los valores p frente a $\alpha = 0.05$ y generar las gráficas de región crítica (χ^2) y distribución acumulada (KS).
- **Etapa 7. Implementación y documentación:** Subir el código al repositorio de GitHub y documentar los resultados en este informe.

4.3. Arquitectura MVC y diseño del software en Python y Java

Ambos proyectos comparten la misma estructura **MVC** dividida en cinco paquetes aplicando *Clean Code* y principios **SOLID** (Tabla 2):

- **Tratamiento numérico (`numberTreatment`):** `NumberGenerator` encapsula el cálculo LCG usando enteros de 64 bits (`long`) en Java para evitar desbordamientos, mientras que `NumberArrayer` recibe el generador por inyección de dependencias y construye el arreglo de $N = 10\,000$ datos.
- **Modelos inmutables (`model`):** `ChiModel`/`Chi` y `KSModel`/`KS` almacenan frecuencias, vectores acumulados, estadísticos, valores p y decisiones mediante `@dataclass` en Python y atributos `private final` en Java.
- **Servicios estadísticos autónomos (`service`):** Sin depender de `scipy`, `AnalyzerChi` calcula el valor p mediante la serie de la función gamma incompleta regularizada superior $Q(gl/2, \chi^2/2)$, y `AnalyzerKS` aplica la serie asintótica de Kolmogorov sobre $z = D\sqrt{N}$.
- **Vistas y controlador (`view`, `controller`):** `View` imprime las tablas en consola; `GraphicsChi` halla $\chi^2_{\text{crítico}}$ por bisección y dibuja la densidad con la zona de rechazo; `GraphicsKS` traza $F_n(x)$ frente a $F(x) = x$; y `Controller` coordina el flujo completo.

Tabla 2: Estructura modular del proyecto APE 006 en Python y Java.

Archivo (Python / Java)	Responsabilidad
`numberTreatment/numberGenerator.py` `numberTreatment/NumberGenerator.java`	Calcula la recurrencia LCG $(aX_n + c) \bmod m$ y normaliza en $[0, 1)$.
`numberTreatment/numberArrayer.py` `numberTreatment/NumberArrayer.java`	Genera y almacena el arreglo de $N = 10\,000$ números.
`model/chi.py` `model/Chi.java`	Guarda frecuencias O_i, E_i, χ^2, gl, valor p y decisión.
`model/KS.py` `model/KS.java`	Guarda datos ordenados, $F_n(x), F(x), D^+, D^-, D$, valor p y decisión.
`service/analyzerChi.py` `service/AnalyzerChi.java`	Agrupa en $k = 10$ clases, calcula χ^2 y su valor p.
`service/analyzerKS.py` `service/AnalyzerKS.java`	Ordena la muestra y calcula D^+, D^-, D y el valor p.
`view/view.py` `view/View.java`	Muestra en consola la secuencia y las tablas de ambas pruebas.
`view/graphicsChi.py` `view/GraphicsChi.java`	Grafica la curva χ^2, la región de rechazo y $\chi^2_{\text{calculado}}$.
`view/graphicsKS.py` `view/GraphicsKS.java`	Grafica la curva escalonada $F_n(x)$ frente a la diagonal $F(x) = x$.
`controller/controller.py` `controller/Controller.java`	Coordina la generación, el análisis estadístico y las vistas.
`main.py` `Main.java`	Punto de entrada que instancia e inyecta las dependencias.

5. Resultados

5.1. Secuencia generada y resultados numéricos

Con $X_0 = 12345$ y $N = 10\,000$, Python y Java generan exactamente la misma secuencia:

- **Primeros 10:** `[0.020403, 0.016548, 0.543156, 0.634904, 0.910030, 0.112462, 0.495889, 0.548349, 0.596100, 0.783450]`
- **Últimos 10:** `[0.549802, 0.782449, 0.368316, 0.657494, 0.932363, 0.969974, 0.559279, 0.390124, 0.052949, 0.650447]`

Las Tablas 3 y 4 resumen el conteo por intervalos y los estadísticos finales.

Tabla 3: Frecuencias por subintervalo de la prueba Chi-cuadrado ($N = 10\,000, k = 10$).

Intervalo	Observada (O_i)	Esperada (E_i)	Diferencia	$(O_i - E_i)^2/E_i$
$[0.0, 0.1)$	1016	1000.00	$+16.00$	0.2560
$[0.1, 0.2)$	993	1000.00	-7.00	0.0490
$[0.2, 0.3)$	1049	1000.00	$+49.00$	2.4010
$[0.3, 0.4)$	940	1000.00	-60.00	3.6000
$[0.4, 0.5)$	949	1000.00	-51.00	2.6010
$[0.5, 0.6)$	1063	1000.00	$+63.00$	3.9690
$[0.6, 0.7)$	1014	1000.00	$+14.00$	0.1960
$[0.7, 0.8)$	1016	1000.00	$+16.00$	0.2560
$[0.8, 0.9)$	984	1000.00	-16.00	0.2560
$[0.9, 1.0]$	976	1000.00	-24.00	0.5760
Total	**10 000**	**10 000.00**	**0.00**	$\boldsymbol{\chi^2 = 14.160000}$

Tabla 4: Resumen de resultados de Chi-cuadrado y Kolmogorov-Smirnov (Python = Java).

Métrica	Chi-cuadrado (χ^2)	Kolmogorov-Smirnov (KS)
Muestra (N)	$10\,000$	$10\,000$
Parámetros	$gl = 9$ ($k = 10$)	$D^+ = 0.005960$ / $D^- = 0.005411$
Estadístico calculado	$\chi^2_{\text{calc}} = 14.160000$	$D = 0.005960$
Valor crítico ($\alpha = 0.05$)	$\chi^2_{\text{crítico}} = 16.9190$	$D_{\text{crítico}} \approx 0.013581$
Valor p	$p = 0.116746$	$p = 0.869508$
Criterio ($p \geq 0.05$)	$0.116746 \geq 0.05$	$0.869508 \geq 0.05$
Decisión	**No se rechaza** H_0	**No se rechaza** H_0

5.2. Salidas en consola (Python y Java)

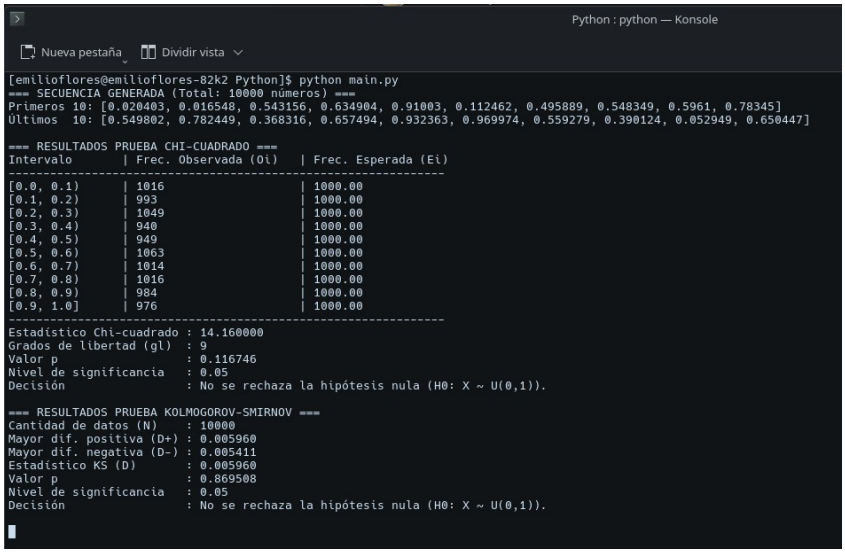


Figura 1: Ejecución en consola en Python mostrando la secuencia generada y los resultados de Chi-cuadrado y Kolmogorov-Smirnov.

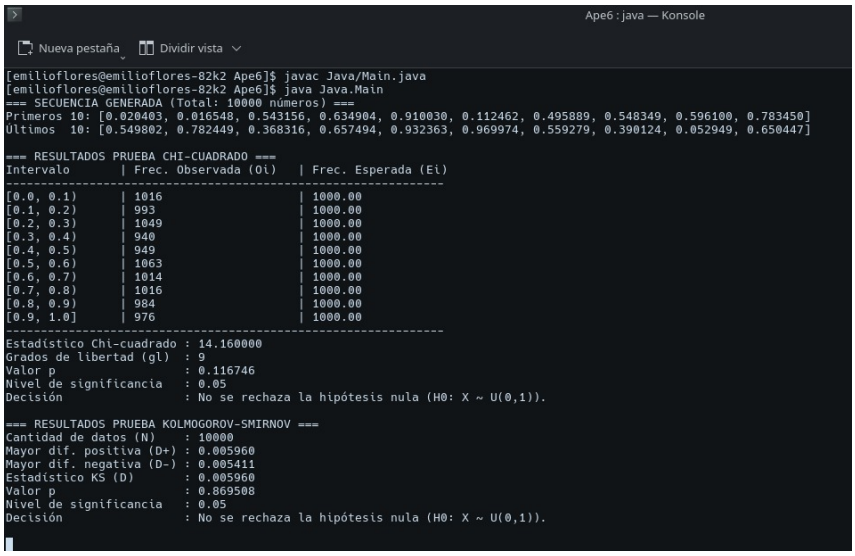


Figura 2: Ejecución en consola en Java mostrando resultados idénticos a la implementación en Python.

5.3. Gráficas de Chi-cuadrado y Kolmogorov-Smirnov (Python y Java)

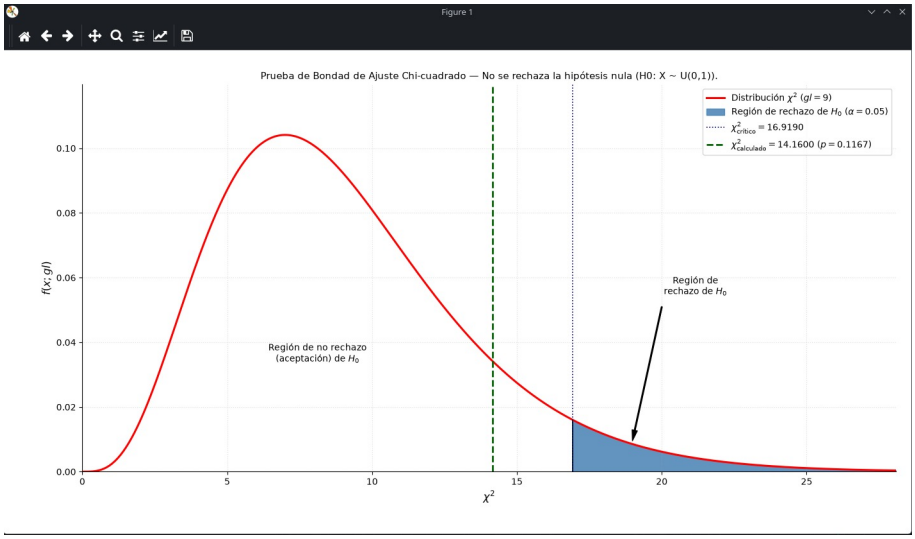


Figura 3: Gráfica de la prueba Chi-cuadrado en Python: curva de densidad ($gl = 9$), región de no rechazo y estadístico $\chi^2_{\text{calc}} = 14.1600$.

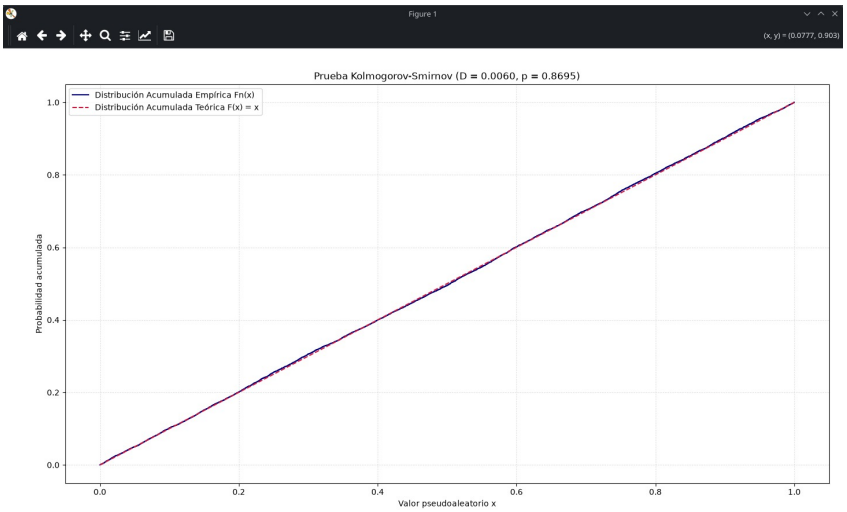


Figura 4: Gráfica de la prueba Kolmogorov-Smirnov en Python: distribución acumulada empírica $F_n(x)$ frente a la teórica $F(x) = x$.

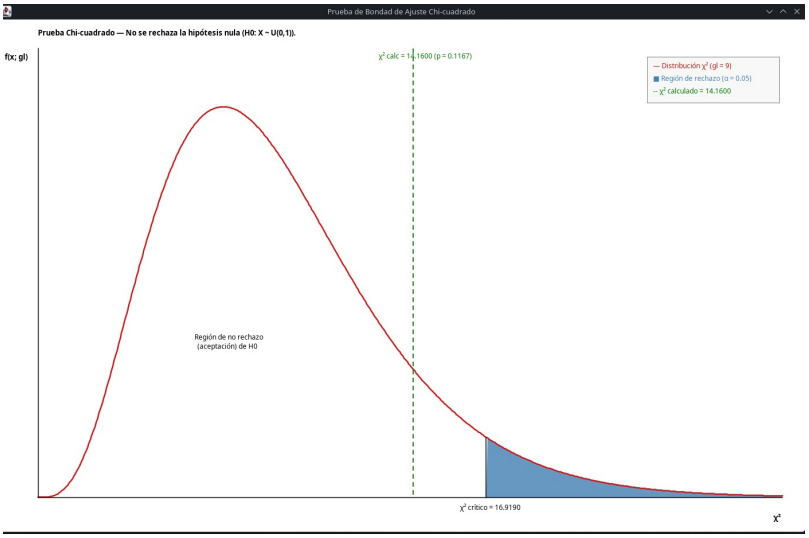


Figura 5: Gráfica de la prueba Chi-cuadrado en Java Swing ($\chi^2_{\text{crítico}} = 16.9190$, $\chi^2_{\text{calc}} = 14.1600$, $p = 0.1167$).

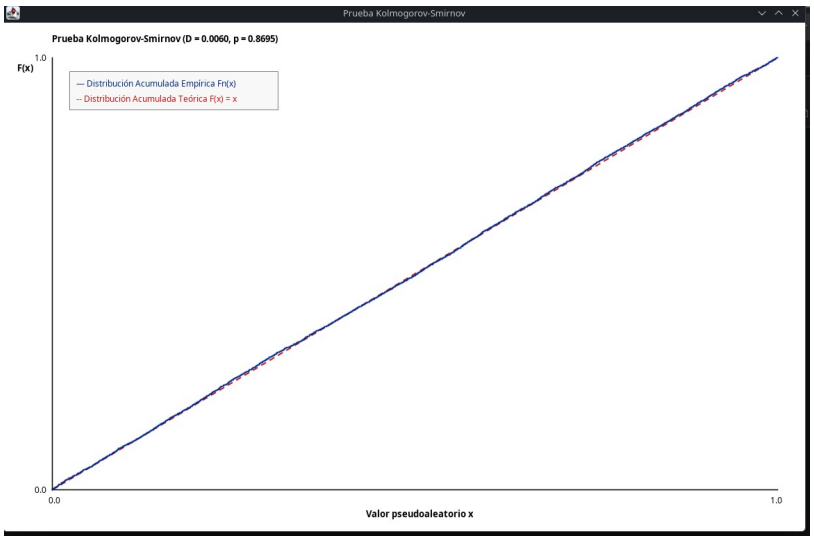


Figura 6: Gráfica de la prueba Kolmogorov-Smirnov en Java Swing ($D = 0.0060$, $p = 0.8695$).

5.4. Análisis de resultados

- **Prueba Chi-cuadrado:** Las frecuencias observadas oscilan cerca de $E_i = 1000$ (entre 940 y 1063). El estadístico $\chi^2_{\text{calc}} = 14.1600$ queda por debajo del crítico (16.9190) dentro de la región de aceptación ($p = 0.116746 \geq 0.05$), por lo que no se rechaza H_0.
- **Prueba Kolmogorov-Smirnov:** La distancia máxima entre $F_n(x)$ y $F(x) = x$ es $D = 0.005960$ (menor al crítico 0.013581), con $p = 0.869508 \geq 0.05$. En las gráficas, la escalonada empírica se superpone a la diagonal teórica, confirmando la uniformidad.
- **Equivalencia Python–Java:** El uso de `long` en Java y las mismas series numéricas producen resultados idénticos hasta el sexto decimal en ambos lenguajes.

5.5. Cumplimiento de resultados esperados

Tabla 5: Verificación de resultados esperados en la práctica APE 006.

Resultado esperado	**Cumplimiento**
Generación LCG de $10\,000$ datos.	Secuencia idéntica en Python y Java (`numberTreatment/`).
Prueba Chi-cuadrado ($k = 10, gl = 9$).	$\chi^2 = 14.160000$ y $p = 0.116746$ (Tabla 3).
Prueba Kolmogorov-Smirnov.	$D = 0.005960$ y $p = 0.869508$ (Tabla 4).
Gráficas de ambas pruebas.	Generadas en `Matplotlib` (Python) y `Swing` (Java).
Decisión con $\alpha = 0.05$.	No se rechaza $H_0 : X \sim U(0,1)$ en ambas pruebas ($p \geq 0.05$).

6. Preguntas de Control

1. **¿Qué es la hipótesis nula?**
   Es el supuesto inicial ($H_0 : X \sim U(0,1)$) que afirma que los números generados siguen una distribución uniforme en $[0,1]$ mientras la muestra no indique lo contrario [1, 3].
2. **¿Qué representa el valor p?**
   Es la probabilidad de obtener un estadístico igual o más extremo que el calculado asumiendo que H_0 es cierta; si $p \geq 0.05$, las desviaciones se atribuyen al azar muestral [2, 3].
3. **¿Qué significa rechazar H_0?**
   Significa que $p < \alpha$ (0.05), por lo que hay evidencia estadística de que la secuencia no es uniforme en $[0,1]$ y el generador debe descartarse [1, 3].
4. **¿Qué diferencia existe entre Chi-cuadrado y KS?**
   Chi-cuadrado agrupa los datos en k intervalos discretos y compara conteos (O_i vs. E_i), mientras que KS evalúa todos los datos individuales ordenados midiendo la distancia máxima entre $F_n(x)$ y $F(x)$ sin agruparlos [3, 4].
5. **¿Por qué una prueba estadística no demuestra que un generador sea verdaderamente aleatorio?**
   Porque el LCG es una ecuación determinista y periódica; superar las pruebas solo confirma uniformidad empírica en la muestra, pero no elimina su predictibilidad ni posibles correlaciones multidimensionales [3, 4].

7. Conclusiones

- Las implementaciones en Python y Java bajo arquitectura MVC generaron exactamente los mismos $10\,000$ valores pseudoaleatorios y los mismos estadísticos de prueba.
- La prueba Chi-cuadrado ($gl = 9$) arrojó $\chi^2 = 14.160000 < 16.9190$ ($p = 0.116746 \geq 0.05$), confirmando uniformidad en las frecuencias por intervalo.
- La prueba Kolmogorov-Smirnov obtuvo $D = 0.005960$ ($p = 0.869508 \geq 0.05$), validando el ajuste de la distribución empírica $F_n(x)$ a la teórica $F(x) = x$.

8. Bibliografía / Referencias

1. J. O. Guamán Q., *Guía de Actividades Práctico-Experimentales Nro. 006: Validación estadística de números pseudoaleatorios mediante Chi-cuadrado y Kolmogorov-Smirnov*. Carrera de Computación, FEIRNNR, Universidad Nacional de Loja, 2026.
2. Asignatura Simulación, *Diapositivas de la Semana 6: Pruebas Estadísticas de Uniformidad e Independencia para Números Pseudoaleatorios*. Carrera de Computación, FEIRNNR - UNL, 2026.
3. A. M. Law, *Simulation Modeling and Analysis*, 5.ª ed. New York, NY, EE. UU.: McGraw-Hill Education, 2014.
4. D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, 3.ª ed. Reading, MA, EE. UU.: Addison-Wesley, 1997.
5. F. J. Massey Jr., "The Kolmogorov-Smirnov test for goodness of fit", *Journal of the American Statistical Association*, vol. 46, no. 253, pp. 68–78, 1951.

9. Anexos

Anexo A. Instalación, compilación y ejecución en Python y Java

Repositorio del proyecto: Repositorio GitHub

```
# 1. Clonar el repositorio
git clone https://github.com/NullDistortion/Repositorio-Simulacion.git

# 2. Ejecutar la implementacion en Python
cd Repositorio-Simulacion/Ape6/Python
python3 -m venv venv
source venv/bin/activate
pip install numpy matplotlib
python main.py

# 3. Compilar y ejecutar la implementacion en Java
cd ../Java
javac Main.java
java Main
```

Anexo B. Declaración de uso de inteligencia artificial

Se declara el uso de herramientas de inteligencia artificial generativa como apoyo para estructurar la arquitectura MVC en Python y Java, programar las series del valor p y generar las vistas gráficas.

Prompt utilizado para el desarrollo de la arquitectura y diseño del código:

> *"Partiendo de la generación de $N = 10000$ números pseudoaleatorios mediante el método congruencial lineal LCG ($X_0 = 12345$, $a = 1664525$, $c = 1013904223$, $m = 2^{32}$) y su validación estadística con las pruebas Chi-cuadrado ($k = 10$ intervalos, $\alpha = 0.05$) y Kolmogorov-Smirnov ($\alpha = 0.05$), implementa el sistema tanto en Python como en Java aplicando una arquitectura Modelo-Vista-Controlador (MVC) separada en los paquetes `numberTreatment`, `model`, `service`, `view` y `controller`, respetando normas de Clean Code y principios SOLID. En `NumberGenerator` y `NumberArrayer` encapsula la recurrencia LCG cuidando usar enteros de 64 bits (`long`) en Java; en `model` define las estructuras inmutables `ChiModel/Chi` y `KSModel/KS`; en `AnalyzerChi` y `AnalyzerKS` calcula los estadísticos χ^2, D^+, D^-, D y sus valores p exactos mediante series de la función gamma incompleta regularizada y de Kolmogorov; en `View` muestra la secuencia generada y las tablas de resultados; y en `GraphicsChi` y `GraphicsKS` grafica la curva de densidad χ^2 con su región de rechazo sombreada y la curva acumulada empírica $F_n(x)$ frente a la teórica $F(x) = x$."*

Declaración de Delegación en Inteligencia Artificial Generativa (Traducción GAIDeT):

Los autores declaran el uso de inteligencia artificial generativa en el proceso de investigación y redacción. De acuerdo con la taxonomía GAIDeT (2026), las siguientes tareas fueron delegadas a herramientas de IAG bajo supervisión humana total:

- Generación de ideas (*Idea generation*)
- Generación de código (*Code generation*)
- Optimización de código (*Code optimization*)
- Automatización de procesos (*Process automation*)
- Creación de algoritmos para análisis de datos (*Creation of algorithms for data analysis*)
- Validación (*Validation*)
- Limpieza de datos (*Data cleaning*)
- Análisis de datos (*Data analysis*)
- Visualización (*Visualization*)

La herramienta de IAG utilizada fue: **Gemini Argon Alto y NotebookLM**.
La responsabilidad del manuscrito final recae íntegramente en los autores.
Las herramientas de IAG no figuran como autores y no asumen responsabilidad por los resultados finales.
Declaración presentada por: Emilio Josue Flores Gallardo.